

NovaADLS: Semi-Supervised Anomaly Detection in Log Series

Nayef Roqaya¹, Hajira Jabeen², and Thorsten Papenbrock¹

¹ Philipps University of Marburg, Marburg, Germany
roqaya@staff.uni-marburg.de, papenbrock@informatik.uni-marburg.de

² University of Cologne, Cologne, Germany
hajira.jabeen@uk-koeln.de

Abstract. Modern IT systems generate large volumes of textual event logs that capture system behavior and are essential for monitoring, fault diagnosis, and reliability. However, anomaly detection in these logs remains challenging, as existing approaches often rely on extensive manual feature engineering, require large amounts of labeled data, or struggle with dynamic and unstable log patterns. In addition, many methods incur significant computational overhead, which limits their practicality for real-time analysis. This study presents NovaADLS, an end-to-end semi-supervised anomaly detection system for textual event series that requires a small set of normal logs for training, but no labeled anomalies. NovaADLS eliminates the need for ground truth labels with a hybrid unsupervised scoring mechanism based on Silhouette analysis and Local Outlier Factor (LOF) that serves to estimate labels for unlabeled training data automatically. Furthermore, NovaADLS removes manual configuration through a universal feature representation and automatic self-configuration mechanism. To address log instability and semantic variations in the event stream, the system integrates semantic embedding features within a hierarchical ensemble learning framework, combining voting and stacking classifiers to leverage complementary strengths of various models rather than relying on a single classifier. Extensive experiments on real-world datasets, including BGL, HDFS, Thunderbird, and Spirit, demonstrate that NovaADLS consistently outperforms state-of-the-art baselines with anomaly detection F1-scores of up to 0.997 and reduced training times from *hours* to *minutes* and inference time from *minutes* to *seconds* compared to previous approaches on both CPU and GPU platforms.

Keywords: Anomaly detection, Semi-supervised learning, Local outlier factor, Silhouette, Unsupervised learning, Textual event series, Ensemble learning

1 Introduction

Anomaly detection (AD) is, in general, essential to build reliable systems [60]. System logs that record key events and runtime states, for example, provide

the data to detect anomalous behaviors. Log-based anomaly detection generally consists of 1) extracting log events and sequences, 2) training a model on these patterns, and 3) classifying new events. These methods learn patterns directly from the data and often provide additional information, such as timestamps, anomaly scores, or severity levels. Despite their effectiveness, current approaches still face practical challenges in real-world deployment.

1. **Feature selection:** Textual event logs are high-dimensional and heterogeneous, combining textual and numerical attributes. Automatically extracting informative features for effective anomaly detection remains a challenge and often requires manual handling due to schema diversity and evolving log formats.
2. **Automatic parameterization:** AD in event sequences based on temporal, semantic and similarity factors, requiring careful tuning of multiple hyperparameters, including search ranges, model architectures, and dimensionality reduction or topic modeling components. Designing an end-to-end approach that automatically configures and adapts these parameters across datasets remains a key challenge.
3. **Robustness to unstable logs:** Unstable logs, distinguished by missing fields, inconsistent structures, or evolving templates, complicate anomaly detection by obscuring the distinction between genuine faults and benign variations. These irregularities increase false positives and delay root cause analysis.
4. **Runtime-Efficiency:** Both training and inference are often time consuming activities in anomaly detection. In practical applications, especially in the inference step, the inference step needs to be able to keep up with the streaming speed of the events to be analyzed. For this reason, anomaly detection requires lightweight training and runtime-efficient inference without compromising accuracy to enable the timely identification of anomalies and minimal operational costs.

To address these four challenges, this study presents NovaADLS, a semi-supervised anomaly detection algorithm for log-based, i.e., textual event series. With NovaADLS, the following contributions are made:

1. An *effective feature set* for semi-supervised based anomaly detection in textual event data.
2. A *self-configuration technique* that automates the parameter tuning and makes the algorithm fully automated in the context of a semi-supervised approach.
3. *Robustness in handling unstable logs* is achieved by incorporating semantic embedding features.
4. An *efficient anomaly detection* architecture that reduces training and testing time while ensuring reliable anomaly detection.

The remainder of this study is organized as follows. Section 2 reviews related work and Section 3 introduces key terminology. Section 4 presents the proposed approach NovaADLS. Section 5 describes the experimental setup and the results. Finally, Section 6 concludes the study and outlines future work.

2 Related Work

This section reviews representative state-of-the-art approaches, with an emphasis on log representation and robustness to evolving log patterns.

Log Data Representation for Anomaly Detection. Recent studies increasingly leverage semantic representations to capture the meaning of log events. SemiSMAC [3] is a semi-supervised anomaly detection framework that leverages large language models for log parsing and pseudo-label generation and improves performance in low-resource settings by expanding labeled data. However, it relies on external language models and pseudo-label quality, and its SMAC-based optimization introduces additional computational overhead, which may limit scalability. The CNN-based method [6] leverages logkey2vec embeddings and convolutional neural networks to automatically learn event relationships from system logs. The model consists of multiple convolutional layers, dropout, and max-pooling, and achieves high detection accuracy, outperforming LSTM and MLP-based methods on HDFS log data. NeuralLog [9] is a log anomaly detection approach that avoids log parsing by directly extracting semantic representations from raw log messages. The method addresses issues caused by parsing errors, such as out-of-vocabulary words and semantic misunderstandings, and achieves high detection performance. Topic modeling with LDA illustrates the effectiveness in organizing runtime data and detecting anomalies [12]. A persistent challenge is log instability caused by evolving logging practices, which introduces unseen events and degrades performance [2, 14]. PLELog [15] addresses this issue by combining word embeddings with TF-IDF aggregation.

Supervised Anomaly Detection. Supervised approaches are based on labeled logs and have been widely studied [18, 19]. Early work explored classical classifiers such as RIPPER, SVMs, Nearest-Neighbor, and Logistic Regression [18, 20], while recent studies incorporate deep learning models, including stacked LSTMs, to improve detection accuracy [2, 21]. Despite strong performance, these methods require extensive labeled data. In contrast, NovaADLS operates in a semi-supervised setting using only a limit amount of normal logs and achieves better or comparable performance to the supervised state-of-the-art such as *NeuralLog* [9], *CNN* [6] and *LogRobust* [2], while significantly reducing the labeling effort.

Unsupervised and Semi-supervised Anomaly Detection. Unsupervised and semi-supervised methods are often more practical for anomaly detection, as they reduce the dependence on labeled data for model training [23, 24, 27–29, 35]. Beyond established techniques such as *DeepLog* [25], *LogAnomaly* [8], and *PLELog* [15], several novel approaches have been introduced. For example, Lou et al. [26] proposes an anomaly detection technique for unstructured logs that automatically discovers program invariants. It first parses logs into structured

form, groups log messages based on parameter relationships, and mines invariants that capture linear characteristics of program workflows. These invariants are then used to detect anomalies. More details on our baselines are provided in Section 5.3

3 Key Terminology in Log Data Processing

A *log message* is an unstructured textual record generated during the execution of a system, application, or service that captures the state of the system at a specific time. *Log parsing* transforms raw messages into structured fields, including *timestamps*, *log events (templates)*, and *log parameters*.

A *log event* corresponds to the invariant textual template defined by developers, while a *log parameter* represents variable system-specific values (e.g., IP addresses). A *log sequence* is a temporally ordered series of log events constructed using identifiers (e.g., task IDs) or sliding windows.

Sequences containing abnormal behavior are labeled anomalous; otherwise, they are considered normal. This work distinguishes between *stable logs*, which follow frequently recurring templates, and *unstable logs*, which originate from rare or evolving templates. This structured representation enables an effective temporal analysis of system behavior.

4 Anomaly Detection with NovaADLS

Anomaly detection (AD) is the process of identifying data elements in a sequence of elements that deviate in value or pattern from the majority of other elements [63]. This section introduces the novel **Nova** semi-supervised **Anomaly Detection in Log Series (NovaADLS)** algorithm¹. Similarly to previous work [15], NovaADLS uses only a small set of labeled normal logs. The overall architecture, shown in Fig. 1, consists of five main phases.

In the following, we provide a detailed description of these phases.

4.1 Log Parsing

The first phase processes input logs in batch or streaming mode, transforming unstructured log messages into structured log events. Raw logs typically contain free-text messages with numerous parameters, making automated analysis difficult [30, 36]. Following prior work [15], NovaADLS uses the Drain parser [32, 36] to extract structured event templates. We perform log parsing using the Drain parser [31], a state-of-the-art log parsing method known for its high accuracy and efficiency, to generate structured log sequences that support reliable and effective downstream analysis. This study validates the quality of the parsing process in Section 5.4.

¹ <https://github.com/nayefroqaya/NovaAD.git>

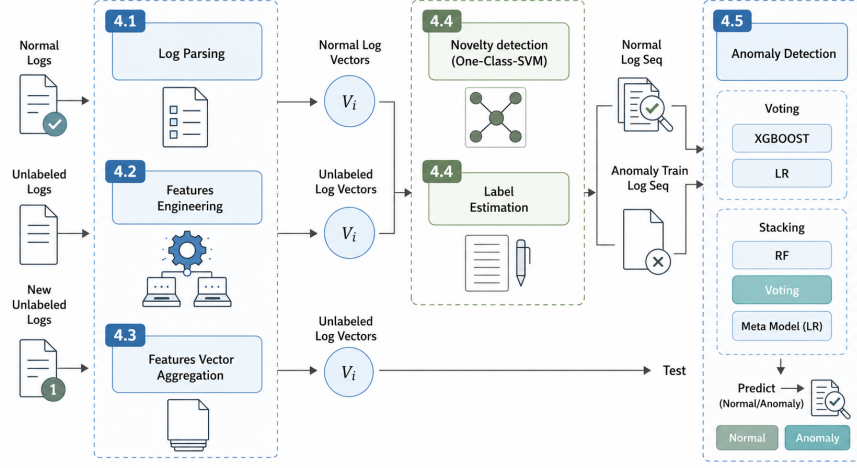


Fig. 1. NovaADLS architecture.

4.2 Feature Engineering

In this phase, NovaADLS extracts, configures, and fine-tunes a set of distinguishing features to facilitate novelty detection and label estimation for unlabeled data within the training set. To ensure compatibility with subsequent processing steps, this proposed approach standardizes and scales the extracted feature vectors. The feature extraction process incorporates various characteristics that provide a comprehensive description of the data. This diverse set of characteristics paints a rich and multidimensional picture of the data, enabling deeper and more precise analyzes. The first group of features, i.e., Dominant Topic Index and Entropy features, is based on topic modeling. Similarly to previous works that use topic-based features for spammer detection in social media posts [33] or anomaly detection in log data [13], the aim is to capture semantic event categories in the form of topics from event texts. Despite the brevity of the texts, they still provide sufficient context for topic modeling. For topic-based features (Dominant Topic Index and Entropy), we apply topic modeling using *Latent Dirichlet Allocation (LDA)* [16] to automatically extract topics from the textual data in each log-event.

Each textual log event E serves as an individual document d . The fundamental concept underlying *LDA* is that a document can be viewed as a composite of various topics, and each topic, in turn, is composed of a specific set of words. These topics exhibit a higher probability of sharing common words. Each document d_l is represented as a multinomial distribution θ_l on a set of latent topics $TP = [tp_1, tp_2, \dots, tp_k]$ with k being the number of topics. Let θ_l be the distribution of the topic of document d_l with $p(tp_j|d_l) = x_{lj}$ representing the probability of topic tp_j in document d_l . Each topic tp_j is also a multinomial distribution ϕ_j on a set of words $W = \{w_1, w_2, \dots, w_m\}$ with m being the number of

words. ϕ_j is defined as $\phi_j = [p(w_1|tp_j), p(w_2|tp_j), \dots, p(w_m|tp_j)]$. By definition, $\sum_{a=1}^m p(w_a|tp_j) = 1$. Latent Dirichlet Allocation (LDA) is a generative probabilistic model commonly used for topic modeling and typically trained on a specific corpus of text data to discover topics within those data. The proposed approach uses text documents as input and fits the model to the data to discover latent topics.

This work treats the textual content of the log events as documents using an LDA model, $D = \{d_1, d_2, \dots, d_n\}$. Using the topic distribution of each document, this phase computes the entropy and dominant topic index for each log event d_l as topic-based features. The number of topics in Latent Dirichlet Allocation (LDA) is tuned through grid search over $n_{\text{components}} \in \{2, 3, 5, 7, 9, 13, 15\}$. We train the model using online variational Bayes with a maximum of 100 iterations and select the optimal topic dimensionality using *GridSearchCV* on the document-term matrix.

1. **Dominant Topic Index Feature (DTIF).** The Dominant Topic Index feature is a topic-based feature derived from Latent Dirichlet Allocation (LDA) [16] that captures the dominant semantic theme of a log event. Each log event d_l is treated as a document and is represented by a topic distribution $Topic(d_l) = [p(tp_1|d_l), p(tp_2|d_l), \dots, p(tp_k|d_l)]$, where $p(tp_j|d_l)$ denotes the probability of topic tp_j . LDA models documents as mixtures of latent topics and provides topic-word probability distributions over the vocabulary. DTIF is defined as follows:

$$DTIF(d_l)_{\text{dominant}} = \arg \max_i p(tp_i|d_l). \quad (1)$$

Here, $\arg \max_i p(tp_i|d_l)$ returns the index of the topic tp_i with the maximum probability of the distribution of the topic. By selecting the topic with the highest-probability, DTIF groups semantically related events despite lexical variation [12], reduces noise from low-probability topics, and improves anomaly detection by highlighting thematic deviations.

2. **Entropy Feature (ENF).** The Entropy feature measures the uncertainty of the topic distribution of a log event obtained from the Latent Dirichlet Allocation (LDA) [33, 34]. Higher entropy indicates that an event spans multiple topics, reflecting greater semantic ambiguity that is often associated with anomalous behavior. ENF therefore complements the dominant-topic feature by highlighting events with mixed or unusual semantic patterns [33]. For a log event d_l with topic distribution $\theta_l = [p(tp_1|d_l), \dots, p(tp_k|d_l)]$, where $x_{lj} = p(tp_j|d_l)$ and k is the number of latent topics, the entropy feature is defined as follows:

$$ENF(d_l) = [Entropy(d_l, tp_1), \dots, Entropy(d_l, tp_k)] \quad (2)$$

$$\text{with } Entropy(d_l, tp_k) = - \sum_{j=1}^K x_{lj} \log_2 x_{lj}.$$

High entropy corresponds to a diffuse topic mixture, whereas low entropy indicates a focused semantic theme.

3. **SBERT Feature (SBTF).** The SBERT feature captures the semantic content of log events using sentence-level embeddings generated by a pre-trained BERT-based Sentence Transformer [59]. Although pre-trained language models such as BERT [7] and GPT [17] are widely used for semantic feature extraction, SBERT [1] is adopted due to its efficiency and suitability for sentence embeddings. SBERT encodes contextual and semantic relationships, enabling a reliable comparison of similarity between log events despite surface-level variations [15]. Specifically, each log event d_l is mapped to a fixed-size embedding $EM_{d_l} = \text{transform}_{BERT}(d_l)$ using a pre-trained *BERT model (bert-base-nli-mean-tokens)*, where $|EM_{d_l}| = 768$. To reduce computational overhead while preserving semantic structure, we apply Principal Component Analysis (PCA) [58] and automatically select the number of components r by maximizing the cumulative explained variance [37, 42]. The final SBERT feature is defined as follows:

$$SBERTF(d_l) = \text{PCA}(\text{transform}_{BERT}(d_l)). \quad (3)$$

4. **Sentiment Feature (SEF).** The Sentiment feature captures the emotional tone of the log event texts, providing complementary contextual cues to handle unstable logs with variable expressions [11, 38]. When combined with Semantic features such as SBERT, Sentiment information improves the robustness to implicit or anomalous system behavior [11]. SEF is computed using a pre-trained model (siebert/sentiment-roberta-large-english) [38], which assigns each log event d_l a binary sentiment label, defined as follows:

$$SEF(d_l) = \text{SentimentRoberta}(d_l). \quad (4)$$

1 denotes positive sentiment and 0 denotes negative sentiment.

5. **Word Count Feature (WCF).** The Word Count feature measures the total number of words in a log message [39]. Log events with unusually high or low word counts often deviate from typical structural patterns and may indicate anomalous behavior. For a log event d_l , the feature is defined as follows:

$$\text{WCF}(d_l) = \sum_{i=1}^n w_i. \quad (5)$$

where w_i denotes each word and n is the total number of words in the message.

6. **Character Count Feature (CCF).** Character Count feature measures the total number of characters in a log message [40]. Log events that are unusually short or long may indicate anomalies such as truncation, excessive verbosity, or unexpected data injection. For a log event d_l , the feature is defined as follows:

$$\text{CCF}(d_l) = \sum_{i=1}^n c_i. \quad (6)$$

where c_i denotes each character and n is the total number of characters.

7. **Temporal Feature (TF).** The temporal feature captures the time-related attributes of log events, including year, month, day, hour, minute, and second [41], allowing the detection of anomalies caused by irregular timing patterns. For a log event d_i , the feature is defined as follows:

$$\text{TF}(d_i) = (\text{Year}, \text{Month}, \text{Day}, \text{Hour}, \text{Minute}, \text{Second}). \quad (7)$$

where each component corresponds to the event timestamp. Higher-level temporal components are particularly informative for datasets with shorter time intervals. For each log event d_i , we combine the seven extracted features into a unified feature vector $\text{Features}(d_i)$ and further configure and transform it.

4.3 Feature Vector Aggregation

This phase constructs log-sequence feature vectors by aggregating log-event features within each sequence using Mean pooling [43]. Mean aggregation normalizes feature values across sequences of varying lengths, providing a stable and robust representation. By averaging event-level features, the approach preserves essential structural patterns while reducing the influence of noise and transient fluctuations, which has been shown to improve robustness in sequence-based anomaly detection [25, 44]. Section 5.4 evaluates the Mean and other aggregation functions (Analysis of the Aggregation Functions).

4.4 Novelty Detection and Label Estimation

This phase identifies log sequences that deviate from normal behavior in a semi-supervised setting using only limited normal training data. After feature aggregation, we apply a One-Class Support Vector Machine (OC-SVM) [45], which learns a decision boundary describing normal system behavior and flags outliers as anomalies *without requiring ground-truth labels* for evaluation as follows.

Stage 1. Hyperparameter Optimization: OC-SVM is governed by two hyperparameters: γ , controlling the influence of individual samples, and ν , balancing the size of the margin and classification errors. NovaADLS selects the hyperparameters using a *Hybrid Score* that combines the local outlier factor (LOF) [46, 47] and the Silhouette [48, 49] scores as follows:

$$\text{Hybrid_Score} = \alpha \cdot \frac{1}{n} \sum_{i=1}^n \text{LOF}_i + (1 - \alpha) \cdot \text{Silhouette}. \quad (8)$$

For each candidate pair (γ, ν) , this phase trains OC-SVM on normal sequences and applies to unlabeled data to predict pseudo-labels. The Hybrid Score is computed and tuned across $\alpha \in [0.5, 1.0]$; the value α^* that maximizes the score is *selected automatically*.

Stage 2. Model Training: Based on α^* , the corresponding pair (γ^*, ν^*) is chosen as the optimal parameters of the OC-SVM. Using (γ^*, ν^*) , this phase

trains the OC-SVM in limited normal sequences to define the boundary of the final decision.

Stage 3. Pseudo-Label Inference: The trained OC-SVM assigns pseudo-labels to unlabeled sequences: decision value -1 indicates anomalies, while all others are normal. This generates labeled data for supervised training, bridging *unsupervised* novelty detection with *supervised* anomaly classification.

4.5 Anomaly Detection

In the final phase, NovaADLS performs anomaly detection using the labeled data produced by the Novelty Detection and Label Estimation stage (Section 4.4), together with a subset of labeled normal data. A supervised hierarchical ensemble framework is adopted to improve classification accuracy and robustness. The ensemble combines a *Voting Classifier* and a *Stacking Classifier* [50, 51]. First, a soft voting scheme aggregates the predicted probabilities from XGBoost [64] and Logistic Regression [20], leveraging complementary nonlinear and linear decision boundaries. The voting ensemble and a Random Forest model [52] are then used as base learners in a stacking architecture, with Logistic Regression as the meta-learner.

Stage 1. Hyperparameter Optimization: In this stage, NovaADLS performs automated hyperparameter optimization using *RandomizedSearchCV* [54] to tune XGBoost and Random Forest. The optimization runs for ten iterations with five-fold stratified cross-validation, optimizing the F_1 score to address class imbalance. NovaADLS uses Logistic Regression with default settings, as further tuning yields negligible improvements [55, 56].

Stage 2. Model Training: Using the optimal hyperparameters, NovaADLS trains the base classifiers with fixed random seeds and constructs a hierarchical ensemble that combines voting outputs and Random Forest predictions via a Logistic Regression meta-model.

Stage 3. Prediction: At inference time, base estimators generate class probabilities, which serve as meta-features for the stacking model to produce predictions. The prediction stage determines the optimal decision boundary by selecting the threshold on the validation set using the precision-recall curve and maximizing the F_1 -score. Then it fixes the selected threshold and applies it to the test set for final evaluation, thus avoiding data leakage. Finally, we evaluated performance using precision, recall, and the F_1 -score.

5 Evaluation

This study deploys the proposed approach on datasets from the log hub [53] for evaluation. The performance of the anomaly detection approach is evaluated in terms of accuracy and runtime through a series of experiments that address the following research questions.

1. **Q1:** How effective is the NovaADLS in terms of *precision*, *recall*, and F_1 score?

2. **Q2:** How efficient is NovaADLS in terms of *runtime*?
3. **Q3:** How robust is NovaADLS in handling unstable logs with evolving patterns and noise?
4. **Q4:** How sensitive are NovaADLS w.r.t. its features measured as the features' *mutual information*?

5.1 Implementations and Environments

The proposed approach performs all experiments offline, implements them in Python 3.11, and conducts them within a unified experimental framework. We reproduce and evaluate the baseline methods LogAnomaly [8], LogRobust [2], CNN [6], NeuralLog [9], DeepLog [25] and PLELog [15] on a dedicated Ubuntu Linux server equipped with an AMD EPYC 7513 CPU (32 cores), 256 GB RAM, and an NVIDIA A10 GPU (24 GB VRAM).

To ensure fair and reproducible comparisons, this study uses identical block-level log sequences with fixed data splits (60% training, 10% validation, 30% testing) for all methods. It conducts training and inference experiments using both CPU- and GPU-accelerated configurations and repeat each setup three times to evaluate result stability.

5.2 Datasets

This study evaluates NovaADLS on five widely used public datasets for log-based anomaly detection: HDFS, BGL, Thunderbird (1G and 2G), and Spirit (150MB), all commonly adopted in previous studies [2, 9, 10]. For datasets without explicit block identifiers (BGL, Thunderbird, and Spirit), log sequences are constructed using a fixed sliding window of size 120, following established practice [8, 15, 25]. For HDFS, sequences are extracted using the dataset's embedded block IDs. HDFS logs capture failures in a distributed file system, while BGL, Spirit, and Thunderbird logs originate from large-scale supercomputing systems with complex event patterns. This diversity ensures that the evaluation reflects both structured and unstructured logging scenarios, as well as varying anomaly distributions.

Following previous work [10, 15, 22], we divide each dataset into train, validate, and test sets using a 6:1:3 ratio. We first order the dataset *chronologically* by block ID and timestamp to preserve the temporal structure of the log events within each block and then partition the data at the block ID level by randomly assigning unique block IDs to the train sets (60%), validate (10%) and test (30%). To prevent *data leakage*, we ensure that block ID sets are mutually exclusive and explicitly verify that no block ID appears in more than one split.

To simulate a semi-supervised setting, we use only a subset of normal log sequences from the train set during training. Specifically, we treat 50% of the normal sequences in the training set as labeled, while we consider the remaining normal sequences and all anomalous sequences as unlabeled, following previous work [15].

5.3 Evaluation Metrics and Baselines

This study evaluates anomaly detection performance using the F_1 -score, precision and recall, which are widely adopted metrics for unbalanced datasets [4], defined as follows:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN} \quad (10)$$

where TP , FP , and FN denote true positives, false positives, and false negatives.

As NovaADLS follows a semi-supervised learning paradigm, the proposed approach is compared with representative state-of-the-art log-based supervised, unsupervised and semi-supervised anomaly detection methods.

CNN [6] proposes a convolutional neural network (CNN)-based approach for anomaly detection in large-scale system logs. Their method leverages logkey2vec embeddings combined with multiple one-dimensional convolutional layers, followed by dropout and max-pooling, to automatically capture event relationships within log sequences.

NeuralLog [9] is a log-based anomaly detection approach that eliminates the need for explicit log parsing. NeuralLog captures the semantic information of the raw log messages and encodes them into semantic vector representations. Subsequently, these vectors are utilized by a Transformer-based classification model to detect anomalies, enabling effective modeling of contextual dependencies within log sequences.

LogRobust [2] transforms log events into semantic vector representations by extracting their underlying meaning. To detect anomalies, it uses an attention-based Bi-LSTM model, which effectively captures contextual relationships within log sequences. The attention mechanism enables the model to automatically weight the significance of different log events. This design allows LogRobust to effectively recognize and manage variability and instability in both individual log events and entire sequences.

DeepLog [25] treats log sequences as natural language sequences and employs a deep neural network, specifically Long Short-Term Memory (LSTM) networks, to learn normal log patterns from typical log sequences. During the anomaly detection process, DeepLog predicts the next log event. If the actual log-event does not appear in the top prediction results, it is considered an anomaly.

LogAnomaly [8] shares similarities with DeepLog in that it also learns normal log patterns from standard log sequences. However, the key difference is that LogAnomaly represents log sequences using semantic information, compared to the one-hot encoding used by DeepLog [25]. This semantic representation improves prediction effectiveness. Additionally, LogAnomaly tracks the frequency of log events during representation, enabling it to detect anomalies based on irregularities in the count of log events.

PLELog [15] is a novel and practical log-based anomaly detection approach that takes advantage of a semi-supervised learning framework to eliminate the

need for manual and time-consuming labeling. PLELog incorporates historical anomaly knowledge through probabilistic label estimation, thereby enhancing the effectiveness of supervised approaches. Additionally, PLELog is robust against unstable log data by utilizing semantic embedding and detects anomalies efficiently and accurately through the design of an attention-based GRU neural network.

5.4 Results and Discussion

To assess the consistency of the Drain log parser [36], we executed the parser *three times* on each dataset using its default configuration. For each run, we recorded the total number of log messages and the numbers of normal and anomalous log sequences. Table 1 summarizes the datasets and the sequencing strategy. The parser produced identical results on all runs, indicating that the sequence extraction process is reproducible and provides reliable inputs for downstream anomaly detection.

To evaluate both *effectiveness and efficiency*, we report the mean and standard deviation of the F_1 -score and the runtime over multiple runs. This allows us to assess the *statistical reliability* of the results, where the low variance indicates consistent performance across different datasets.

Table 1. Summary of log datasets and extracted sequences

Dataset	Logs	Normal Seq.	Anom. Seq.	Seq.
BGL	4,747,963	48,652	36,274	Sliding
HDFS	11,175,629	558,223	16,838	BlockID
Thunderbird 1G	6,013,029	44,732	7,601	Sliding
Thunderbird 2G	13,451,157	105,532	9,095	Sliding
Spirit 150MB	1,225,059	5,712	4,792	Sliding

Q1 – Effectiveness of NovaADLS. To evaluate the effectiveness and robustness of NovaADLS in controlled and reproducible settings, we compare NovaADLS with representative supervised, unsupervised, and semi-supervised baselines using fixed block-level log sequences and standardized metrics. All experiments are repeated *three times*; *stability* is measured by the *standard deviation* (std) of F1. Table 2 and 3 report F1 score, precision, recall and F1_std on five benchmark datasets from different sources. *To better reflect differences in dataset sources*, we present the results in two tables. Table 2 reports the results on HDFS and BGL, while Table 3 presents the results on Thunderbird and Spirit, which originate from large-scale high-performance computing systems at *Sandia National Laboratories*.

Among supervised methods, CNN and LogRobust perform strongly, with CNN achieving the highest F1 scores on several datasets, while NeuralLog shows

Table 2. Anomaly detection results (Precision / Recall / F1 $\pm$ standard deviation (std)), averaged over three runs for BGL and HDFS datasets.

Cat.	Method	HDFS	BGL
Sup.	LogRobust	.933/.949/.942 $\pm$.00	.984/.903/.941 $\pm$.1
	CNN	.960/.987/.974 $\pm$.00	.997/.994/.996 $\pm$.00
	NeuralLog	.394/.325/.354 $\pm$.00	.926/.827/.874 $\pm$.2
Unsup.	LogAnomaly	.985/.233/.382 $\pm$.00	.997/.719/.821 $\pm$.12
	DeepLog	.984/.234/.380 $\pm$.00	.996/.731/.830 $\pm$.14
Semi.	PLELog	.967/.723/.827 $\pm$.00	.701/.986/.842 $\pm$.03
	NovaADLS	.991/.998/.994$\pm$.00	.928/.986/.956$\pm$.00

Table 3. Anomaly detection results (Precision / Recall / F1 $\pm$ standard deviation (std)), averaged over three runs for Thunderbird dataset and Spirit.

Cat.	Method	TH-1G	TH-2G	SP-150
Sup.	LogRobust	.998/.994/.996 $\pm$.00	.998/.989/.993 $\pm$.00	.987/.960/.973 $\pm$.00
	CNN	.999/.996/.998 $\pm$.00	.998/.990/.994 $\pm$.00	1.0/.967/.983 $\pm$.00
	NeuralLog	.997/.992/.995 $\pm$.00	.998/.988/.99 $\pm$.00	.902/.857/.879 $\pm$.02
Unsup.	LogAnomaly	.247/.0029/.005 $\pm$.00	.527/.011/.022 $\pm$.00	.978/1.0/.988 $\pm$.00
	DeepLog	.216/.003/.006 $\pm$.00	.140/.015/.015 $\pm$.11	.956/1.0/.978 $\pm$.00
Semi.	PLELog	.286/.985/.441 $\pm$.02	.198/.992/.331 $\pm$.00	.869/.857/.860 $\pm$.04
	NovaADLS	1.0/.996/.997$\pm$.00	1.0/.987/.993$\pm$.00	.918/.950/.932$\pm$.00

greater variance on HDFS. Unsupervised methods (LogAnomaly and DeepLog) exhibit limited robustness, particularly in Thunderbird, where recall drops sharply. The semi-supervised baseline PLELog improves recall, but remains inconsistent across datasets. In contrast, NovaADLS achieves consistently high F1 scores with low variance in all datasets. Although occasionally slightly below fully supervised models for BGL and Spirit, it significantly outperforms supervised baselines for HDFS and Thunderbird dataset, unsupervised and semi-supervised baselines for all datasets, demonstrating a strong balance between accuracy and robustness without the need for extensive labeled data.

Analysis of the Aggregation Functions. Log sequences consist of a variable number of events represented by fixed-dimensional feature vectors that require aggregation into a sequence-level representation. Classical approaches, such as PLELog [15], use Sum aggregation, which can introduce bias when sequence lengths vary. In contrast, Deep Sets [62] provide a theoretical foundation for permutation-invariant aggregation and promote normalized functions, such as the *Mean*. Statistical aggregation has been widely adopted in anomaly detection, including spacecraft telemetry monitoring [61] and LogBERT [57], highlighting the importance of sequence-level representations. Motivated by these insights, *Mean aggregation* is adopted, which is length-invariant and produces stable sequence representations. Empirically, it consistently improves precision and F1

score across multiple datasets in our AD algorithm evaluations. For example, on the BGL dataset, using Mean aggregation, NovaADLS achieves a F1 score of 0.973, compared to 0.859 when using Sum aggregation, with a substantial improvement in precision (0.971 vs. 0.764). In the Spirit (SP) dataset, the Mean aggregation similarly improves performance, producing a F1 score of 0.944 compared to 0.896 with the sum aggregation. In the Thunderbird (1G) dataset, the Mean aggregation achieves a F1 score of 0.996, slightly higher than 0.995 obtained with the Sum aggregation, with identical precision (1.0) and improved recall (0.992 vs. 0.991). Overall, these results demonstrate that Mean aggregation produces more stable and generalizable sequence representations across datasets.

Consideration of the Pseudo-labeling Approach OC-SVM. In NovaADLS, we intentionally use the OC-SVM as a *recall-oriented* novelty detector rather than a high-precision classifier. Its primary role is to maximize the coverage of anomalous patterns during pseudo-label generation, even at the cost of introducing label noise. Consequently, low pseudo-label precision does not contradict the strong final detection performance. The subsequent supervised ensemble learns robust decision boundaries and suppresses noisy labels, thereby reducing noise in the pseudo-labeled data. This design allows NovaADLS to benefit from high anomaly recall at the novelty detection stage while achieving high precision and F1 scores in the final prediction. OC-SVM, for example, achieves high recall on pseudo-labeled training data across datasets (0.99 in BGL, 0.95 in SP (150MB) and 1.0 in Thunderbird (1G), confirming its effectiveness in capturing anomalous patterns prior to *supervised refinement*.

Comparative Analysis of Single and Ensemble Classifiers. To assess the benefits of assembly, this study compares a single *the XGBClassifier* with an ensemble composed of *the XGBClassifier*, *the RandomForestClassifier*, and *LogisticRegression* (the *Final Models*). For example, on the SP (150MB) dataset, the *single* classifier achieves Precision = 0.907, Recall = 0.917 and F1 score = 0.912, while the ensemble improves these metrics to Precision = 0.933, Recall = 0.975, and F1 score = 0.953. The ensemble outperforms the single classifier in all metrics, with a particularly notable increase in Recall (7.5%), indicating that the *ensemble* is more effective in detecting anomalies. This improvement arises because the ensemble combines the complementary strengths of multiple models: XGB captures complex non-linear relationships, RandomForest provides robustness to overfitting, and Logistic Regression offers stable linear decision boundaries. By aggregating these predictions, the ensemble mitigates the weaknesses of the individual model, resulting in more accurate and robust anomaly detection in the Spirit dataset (SP).

Q2 – Efficiency of NovaADLS. Beyond detection accuracy, the practical implementation of log anomaly detection methods is strongly dependent on computational efficiency. Tables 4 and 5 report the average training and testing runtime

(mean $\pm$ std over three runs) of NovaADLS and the baseline methods on the CPU, while Tables 6 and 7 present the corresponding results on the GPU in five benchmark datasets.

CPU Runtime. Fully supervised methods (LogRobust, CNN, and NeuralLog) incur substantial training overhead on large datasets, particularly HDFS and Thunderbird, with NeuralLog exceeding a thousand minutes (1163 minutes) for training in HDFS. Unsupervised methods exhibit mixed behaviors: DeepLog achieves very low runtimes but suffers from limited detection capabilities; LogAnomaly shows poor efficiency, especially in Thunderbird (1G and 2G); the semi-supervised baseline PLELog reduces training costs relative to supervised models, but still incurs non-negligible training and inference overheads; NovaADLS, in contrast, maintains consistently low CPU runtime on all datasets, with training times below 25 min and near-constant inference latency (approximately 0.01–0.06 min). The low standard deviation further indicates a stable and predictable runtime behavior.

Table 4. CPU runtime (Train / Test $\pm$ standard deviation (std) in training time, minutes), averaged over three runs (HDFS, BGL).

Cat.	Method	HDFS	BGL
Sup.	LogRobust	266.0/.49 $\pm$.02	14.6/.37 $\pm$.00
	CNN	314.0/.35 $\pm$.03	15.5/.27 $\pm$.00
	NeuralLog	1163.0/.58 $\pm$.02	72.9/.52 $\pm$.06
Unsup.	LogAnomaly	117.0/.04 $\pm$.00	7.5/.11 $\pm$.01
	DeepLog	7.3/.48 $\pm$.01	4.1/.04 $\pm$.00
Semi.	PLELog	65.0/.72 $\pm$.03	29.0/.26 $\pm$.00
	NovaADLS	22.9/.06$\pm$.00	3.06/.01$\pm$.01

Table 5. CPU runtime (Train / Test $\pm$ standard deviation (std) in training time, minutes), averaged over three runs (Thunderbird, Spirit).

Cat.	Method	TH-1G	TH-2G	SP-150M
Sup.	LogRobust	17.4/.41 $\pm$.01	39.6/1.01 $\pm$.00	3.9/.10 $\pm$.00
	CNN	22.58/.35 $\pm$ 2.0	49.73/.86 $\pm$.3	3.9/.08 $\pm$.00
	NeuralLog	89.1/.62 $\pm$.02	210.0/1.52 $\pm$.06	20.3/.17 $\pm$.02
Unsup.	LogAnomaly	179.0/1.20 $\pm$.02	697.0/4.42 $\pm$.20	5.3/.06 $\pm$.00
	DeepLog	9.0/.04 $\pm$.00	20.5/.12 $\pm$.01	1.6/.01 $\pm$.00
Semi.	PLELog	23.7/.27 $\pm$.01	51.2/.59 $\pm$.00	4.8/.05 $\pm$.00
	NovaADLS	2.15/.01$\pm$.00	4.64/.02$\pm$.00	1.9/.01$\pm$.00

GPU Runtime. GPU acceleration substantially reduces the training times for computationally intensive models (e.g., NeuralLog), making them more feasible at scale. Unsupervised methods achieve very fast execution on GPU, though efficiency issues persist for LogAnomaly on large datasets such as Thunderbird. PLELog benefits from moderate speedups. NovaADLS shows minimal differences between CPU and GPU runtimes, indicating an efficient, hardware-agnostic design. Training times remain below 25 min and inference times near 0.01 min between datasets.

In general, NovaADLS consistently provides fast training, low-latency inference, and stable runtime on both CPU and GPU platforms, demonstrating strong efficiency and practicality for large-scale log anomaly detection deployments.

Table 6. GPU runtime (Train / Test $\pm$ standard deviation (std) in training time, minutes), averaged over three runs (HDFS, BGL).

Cat.	Method	HDFS	BGL
Sup.	LogRobust	165.0/.37 $\pm$.00	7.5/.27 $\pm$ 1.1
	CNN	271.0/.29 $\pm$.02	13.9/.24 $\pm$.02
	NeuralLog	174.0/.36 $\pm$.04	8.8/.28 $\pm$ 1.0
Unsup.	LogAnomaly	34.97/.04 $\pm$.00	0.66/.01 $\pm$.02
	DeepLog	1.95/.05 $\pm$.00	0.53/.01 $\pm$.10
Semi.	PLELog	25.1/.45 $\pm$.01	8.7/.18 $\pm$.01
	NovaADLS	21.1/.06$\pm$.01	2.58/.01$\pm$.00

Table 7. GPU runtime (Train / Test $\pm$ standard deviation (std) in training time, minutes), averaged over three runs (Thunderbird, Spirit).

Cat.	Method	TH-1G	TH-2G	SP-150M
Sup.	LogRobust	9.7/.31 $\pm$.01	21.8/.71 $\pm$.00	2.0/.07 $\pm$.00
	CNN	18.3/.28 $\pm$.2	44.65/.67 $\pm$.07	3.7/.07 $\pm$.01
	NeuralLog	11.1/.29 $\pm$.03	24.9/.01 $\pm$.03	2.3/.08 $\pm$.00
Unsup.	LogAnomaly	13.1/.11 $\pm$.00	46.5/.39 $\pm$.01	0.44/.01 $\pm$.00
	DeepLog	4.02/.09 $\pm$.05	4.9/.02 $\pm$.01	0.23/.01 $\pm$.00
Semi.	PLELog	10.7/.22 $\pm$.01	23.4/.48 $\pm$.00	2.2/.04 $\pm$.00
	NovaADLS	2.06/.01$\pm$.00	4.2/.01$\pm$.01	1.9/.02$\pm$.00

Q3 – Robustness of NovaADLS with Stable and Unstable Log Template. This section discusses the performance of NovaADLS in the BGL dataset under varying proportions of *stable* and *unstable* log templates. When evaluated in fully stable logs, the model achieves a solid baseline performance (Precision = 0.842, Recall = 0.852, F1 = 0.847). In particular, introducing unstable templates

consistently improves robustness: a balanced configuration with 50% stable and 50% unstable templates improves overall performance (Precision = 0.892, Recall = 0.842, F1 = 0.866), indicating that increased template diversity does not affect detection and can improve anomaly discrimination. The strongest results are observed when stable templates remain dominant while allowing moderate instability (75% stable / 25% unstable), achieving Precision = 0.877, Recall = 0.907, and F1 = 0.892. These findings demonstrate that the proposed method handles evolving and infrequent log patterns effectively, maintaining strong detection performance even in the presence of substantial log instability. Overall, the results confirm the robustness of our approach to unstable logs commonly encountered in real-world systems.

Q4 – Ablation Study for NovaADLS’s Features. Feature relevance is evaluated using Mutual Information(MI) [5], a non-linear measure of dependency between a feature X and the target variable Y , defined as:

$$MI(X; Y) = \sum_{x,y} P(x,y) \log \frac{P(x,y)}{P(x)P(y)}. \quad (11)$$

We scaled the MI scores for visualization without affecting their relative rankings. Fig. 2 reports MI-based feature importance in five datasets (BGL, HDFS, TH_1G, TH_2G, SP_150MB) for semantic, statistical/lexical, sentiment and temporal features.

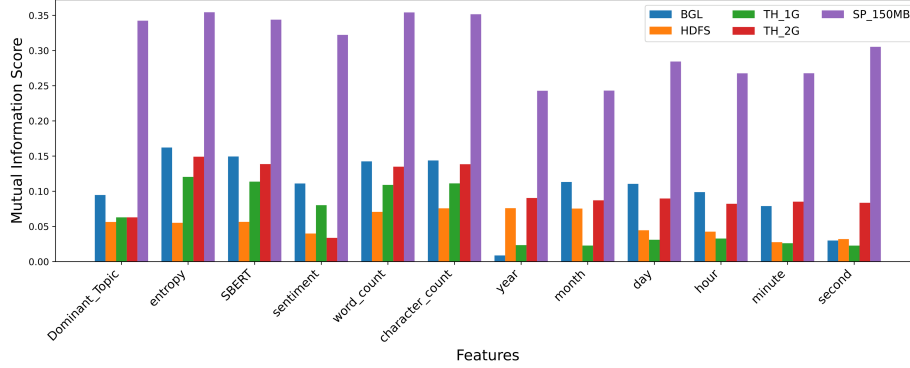


Fig. 2. Mutual Information-based feature importance across datasets.

Across datasets, *Entropy*, *SBERT* embeddings, and *word* and *Character Counts* achieve the highest MI scores, indicating a strong discriminative capability. The *Dominant Topic Index* contributes meaningful coarse-grained semantics, while *Sentiment* provides complementary information. *Temporal* features show dataset-dependent behavior, with limited importance in smaller datasets but

increased relevance in larger and more dynamic workloads. *SBERT* maintains consistently high MI across all datasets, demonstrating robustness and generalizability between datasets. Although absolute MI values are small – as expected in high-dimensional log data – the results show that anomaly detection benefits from the joint contribution of multiple feature families, motivating the proposed hybrid feature design.

Discussion. The experimental results show that the hierarchical ensemble consistently outperforms individual classifiers. This is because it leverages complementary strengths and, in this way, improves both accuracy and robustness. Furthermore, we find that the pseudo-labeling strategy and the hybrid scoring mechanism applied to the OC-SVM outputs (without ground truth) yield a more reliable label estimation than in related works. Our in-depth evaluations also found that Mean aggregation outperforms Sum aggregation in our setup and is, therefore, used by default. With respect to efficiency, NovaADLS achieves significant runtime reductions for the training and inference on both CPU and GPU platforms. The performance is robust for stable and unstable log templates, which shows the adaptability of the approach to dynamic log patterns.

6 Conclusion

In this study, we introduced NovaADLS, an end-to-end semi-supervised framework for anomaly detection in textual event logs that requires only a small set of normal log elements and does not depend on labeled anomalies. The approach combines diverse feature representations within a hierarchical ensemble learning architecture and uses a hybrid unsupervised scoring mechanism to remove the need for ground truth labels in intermediate stages. Furthermore, its automatic self-configuration capability minimizes manual intervention and improves adaptability to evolving log patterns. Experiments on multiple real-world datasets show that NovaADLS achieves a consistently strong detection performance and reduces computation costs compared to state-of-the-art anomaly detection methods. Hence, NovaADLS is an effective and efficient solution for real-time log analysis.

Future Work. The evaluation follows the common setup of evaluations of log-based anomaly detection approaches and is limited to offline benchmark datasets, which may not fully reflect real-world deployment conditions. Our future work will, therefore, focus not only on further improving detection accuracy and efficiency but also on enhancing scalability for large-scale and streaming log data. A possible direction would be the integration of NovaADLS into a distributed processing framework, such as PySpark.

Acknowledgments. The authors appreciate the support and guidance of Lin Yang, from Tianjin University, whose input has been instrumental in the development of this work.

References

1. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3982–3992 (2019). <https://api.semanticscholar.org/CorpusID:201646309>
2. Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z., Chen, J., He, X., Yao, R., Lou, J.-G., Chintalapati, M., Shen, F., Zhang, D.: Robust log-based anomaly detection on unstable log data. In: *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2019)*, pp. 807–817. ACM, New York (2019). <https://api.semanticscholar.org/CorpusID:191140040>
3. Sun, Y., Keung, J.W., Yang, Z., Liu, S., Liao, Y.: SemiSMAC: a semi-supervised framework for log anomaly detection with automated hyperparameter tuning. *Inf. Softw. Technol.* 187, 107869 (2025). <https://doi.org/10.1016/j.infsof.2025.107869>
4. Powers, D.M.W.: Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation. *CoRR abs/2010.16061* (2020). <https://api.semanticscholar.org/CorpusID:3770261>
5. Mroueh, Y., Melnyk, I., Dognin, P., Ross, J., Sercu, T.: Improved mutual information estimation. In: *Advances in Neural Information Processing Systems (NeurIPS) Workshop* (2020). <https://openreview.net/forum?id=S1lsICEYPB>
6. Lu, S., Wei, X., Li, Y., Wang, L.: Detecting anomaly in big data system logs using convolutional neural network. In: *Proc. IEEE Int. Conf. on Dependable, Autonomic and Secure Computing / Pervasive Intelligence and Computing / Big Data Intelligence and Computing / Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*, pp. 151–158. IEEE (2018). <https://api.semanticscholar.org/CorpusID:53072954>
7. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pp. 4171–4186 (2019). <https://api.semanticscholar.org/CorpusID:52967399>
8. Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P., Zhou, R.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence (IJCAI 2019)*, pp. 4739–4745. International Joint Conferences on Artificial Intelligence Organization (2019). <https://api.semanticscholar.org/CorpusID:199466044>
9. Le, V.-H., Zhang, H.: Log-based anomaly detection without log parsing. In: *Proc. IEEE/ACM Int. Conf. on Automated Software Engineering (ASE)*, pp. 492–504. IEEE (2021). <https://doi.ieeecomputersociety.org/10.1109/ASE51524.2021.9678773>
10. He, S., Zhu, J., He, P., Lyu, M.R.: Experience report: System log analysis for anomaly detection. In: *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE)*, pp. 207–218. IEEE (2016). <https://doi.org/10.1109/ISSRE.2016.21>
11. Egersdoerfer, C., Zhang, D., Dai, D.: ClusterLog: clustering logs for effective log-based anomaly detection. In: *Proc. IEEE/ACM Workshop on Fault Tolerance for HPC at eXtreme Scale (FTXS)*, pp. 1–10. IEEE (2022). <https://doi.ieeecomputersociety.org/10.1109/FTXS56515.2022.00006>

12. Steinhauer, H.J., Helldin, T., Mathiason, G., Karlsson, A.: Topic Modeling for Anomaly Detection in Telecommunication Networks. *Journal of Ambient Intelligence and Humanized Computing*, 14(11), 15085–15096 (2023). <https://doi.org/10.1007/s12652-019-01372-5>
13. Viola, L., Ronchieri, E., Cavallaro, C.: Combining Log Files and Monitoring Data to Detect Anomaly Patterns in a Data Center. *Computers*, 11(8), 117 (2022). <https://doi.org/10.3390/computers11080117>
14. Kabinna, S., Bezemer, C.-P., Shang, W., Syer, M.D., Hassan, A.E.: Examining the stability of logging statements. *Empir. Softw. Eng.* 23(1), 290–333 (2018). <https://doi.org/10.1007/s10664-017-9518-0>
15. Yang, L., Chen, J., Wang, Z., Wang, W., Jiang, J., Dong, X., Zhang, W.: PLELog: semi-supervised log-based anomaly detection via probabilistic label estimation. In: *Proc. IEEE/ACM Int. Conf. on Software Engineering: Companion (ICSE-Companion)*, pp. 230–231. IEEE (2021). <https://api.semanticscholar.org/CorpusID:235639082>
16. Blei, D.M., Ng, A.Y., Jordan, M.I.: Latent Dirichlet allocation. *J. Mach. Learn. Res.* 3, 993–1022 (2003). <http://portal.acm.org/citation.cfm?id=944937>
17. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Proc. 31st International Conference on Neural Information Processing Systems (NIPS 2017)*, pp. 6000–6010. Curran Associates, Red Hook, NY (2017). <https://api.semanticscholar.org/CorpusID:13756489>
18. Liang, Y., Zhang, Y., Xiong, H., Sahoo, R.: Failure prediction in IBM BlueGene/L event logs. In: *Proc. IEEE Int. Conf. on Data Mining (ICDM)*, pp. 583–588. IEEE (2007). <https://api.semanticscholar.org/CorpusID:14466898>
19. Breier, J., Branisová, J.: A dynamic rule creation based anomaly detection method for identifying security breaches in log records. *Wirel. Pers. Commun.* 94, 497–511 (2015). <https://doi.org/10.1007/s11277-015-3128-1>
20. Zou, X., Hu, Y., Tian, Z., Shen, K.: Logistic regression model optimization and case analysis. In: *2019 IEEE 7th International Conference on Computer Science and Network Technology (ICCSNT)*, pp. 135–139. IEEE (2019). <https://doi.org/10.1109/ICCSNT47585.2019.8962457>
21. Zhang, K., Xu, J., Min, M.R., Jiang, G., Pelechris, K., Zhang, H.: Automated IT system failure prediction: a deep learning approach. In: *Proc. IEEE Int. Conf. on Big Data*, pp. 1291–1300. IEEE (2016). <https://doi.ieeecomputersociety.org/10.1109/BigData.2016.7840733>
22. Vinayakumar, R., Soman, K.P., Poornachandran, P.: Long short-term memory based operation log anomaly detection. In: *Proc. IEEE Int. Conf. on Advances in Computing, Communications and Informatics (ICACCI)*, pp. 236–242. IEEE (2017). <https://doi.org/10.1109/ICACCI.2017.8125846>
23. Jiang, H., Li, X., Yang, Z., Xuan, J.: What causes my test alarm? Automatic cause analysis for test alarms in system and integration testing. *CoRR abs/1703.00768* (2017). <https://doi.org/10.1109/ICSE.2017.71>
24. Hamooni, H., Debnath, B., Xu, J., Zhang, H., Jiang, G., Mueen, A.: LogMine: fast pattern recognition for log analytics. In: *Proc. ACM Int. Conf. on Information and Knowledge Management (CIKM)*, pp. 1573–1582. ACM (2016). <https://api.semanticscholar.org/CorpusID:5406097>
25. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS 2017)*, pp. 1285–1298. ACM, New York (2017). <https://doi.org/10.1145/3133956.3134015>

26. Lou, J.-G., Fu, Q., Yang, S., Xu, Y., Li, J.: Mining invariants from console logs for system problem detection. In: Proc. USENIX Annu. Tech. Conf., p. 24. USENIX Association (2010). <https://api.semanticscholar.org/CorpusID:17985759>
27. He, S., Lin, Q., Lou, J.-G., Zhang, H., Lyu, M.R., Zhang, D.: Identifying impactful service system problems via log analysis. In: Proc. ACM Joint Meeting on European Software Engineering Conf. and Symp. on the Foundations of Software Engineering (ESEC/FSE), pp. 60–70. ACM (2018). <https://doi.org/10.1145/3236024.3236083>
28. Vigna, G., Valeur, F., Balzarotti, D., Robertson, W., Kruegel, C., Kirda, E.: Reducing errors in the anomaly-based detection of web-based attacks through the combined analysis of web requests and SQL queries. *J. Comput. Secur.* 17(3), 305–329 (2009). <https://api.semanticscholar.org/CorpusID:8463078>
29. Li, T., Jiang, Y., Zeng, C., Xia, B., Liu, Z., Zhou, W., Zhu, X., Wang, W., Zhang, L., Wu, J., Xue, L., Bao, D.: FLAP: an end-to-end event log analysis platform for system management. In: Proc. ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, pp. 1547–1556. ACM (2017). <https://doi.org/10.1145/3097983.3098022>
30. He, P., Zhu, J., He, S., Li, J., Lyu, M.R.: Towards automated log parsing for large-scale log data analysis. *IEEE Trans. Dependable Secure Comput.* 15(6), 931–944 (2018). <https://api.semanticscholar.org/CorpusID:51692559>
31. He, P., Zhu, J., He, S., Li, J., Lyu, M.R.: An evaluation study on log parsing and its use in log mining. In: Proc. IEEE/IFIP Int. Conf. on Dependable Systems and Networks (DSN), pp. 654–661. IEEE (2016). <https://api.semanticscholar.org/CorpusID:9995103>
32. Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z., Lyu, M.R.: Tools and benchmarks for automated log parsing. In: Proc. Int. Conf. on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp. 121–130. IEEE (2019). <https://doi.org/10.1109/ICSE-SEIP.2019.00021>
33. Koggalahewa, D., Xu, Y., Foo, E.: An Unsupervised Method for Social Network Spammer Detection Based on User Information Interests. *Journal of Big Data*, 9(1), 1–35 (2022). <https://doi.org/10.1186/s40537-021-00552-5>
34. Liu, L., Lu, Y., Luo, Y., Zhang, R., Itti, L., Lu, J.: Detecting smart spammers on social networks: a topic model approach. *CoRR abs/1604.08504* (2016). <https://aclanthology.org/N16-2007/>
35. Hassani, M., Shang, W., Shihab, E., Tsantalis, N.: Studying and detecting log-related issues. *Empir. Softw. Eng.* 23(6), 3248–3280 (2018). <https://doi.org/10.1007/s10664-018-9603-z>
36. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: an online log parsing approach with fixed depth tree. In: Proc. IEEE Int. Conf. on Web Services (ICWS), pp. 33–40. IEEE (2017). <https://api.semanticscholar.org/CorpusID:4776668>
37. Gupta, A., Barbu, A.: Parameterized principal component analysis. *Pattern Recognit.* 78, 215–227 (2018). <https://www.sciencedirect.com/science/article/pii/S0031320318300268>,
38. Heitmann, M., Siebert, C., Hartmann, J., Schamp, C.: More than a feeling: benchmarks for sentiment analysis accuracy. SSRN 3489963 (2020). <https://www.sciencedirect.com/science/article/pii/S0167811622000477>
39. Mishra, A.K., Bagla, P., Sharma, R., Pandey, N.K., Tripathi, N.: Anomaly detection from web log data using machine learning model. In: Proc. Int. Conf. on Computer Applications in Electrical Engineering – Recent Advances (CERA), pp. 1–6. IEEE (2023). <https://doi.org/10.1109/CERA59325.2023.10455153>
40. Mohaghegh, M., Abdurakhmanov, A.: Anomaly detection in text data sets using character-level representation. *J. Phys.: Conf. Ser.* 1880, 012028 (2021). <https://iopscience.iop.org/article/10.1088/1742-6596/1880/1/012028>

41. Liu, Y., Ren, S., Wang, X., Zhou, M.: Temporal logical attention network for log-based anomaly detection in distributed systems. *Sensors* 24(24), 7949 (2024). <https://doi.org/10.3390/s24247949>
42. Greenacre, M., Groenen, P., Hastie, T., Iodice d'Enza, A., Markos, A., Tuzhilina, E.: Principal component analysis. *Nat. Rev. Methods Primers* 2, 100 (2022). https://doi.org/10.1007/978-0-387-73003-5_479
43. Ali, S., Boufaied, C., Bianculli, D., Branco, P., Briand, L.: A comprehensive study of machine learning techniques for log-based anomaly detection. *Empir. Softw. Eng.* 30(5) (2025). <https://doi.org/10.1007/s10664-025-10669-3>
44. Wu, X., Li, H., Khomh, F.: On the effectiveness of log representation for log-based anomaly detection. *Empir. Softw. Eng.* 28(6) (2023). <https://doi.org/10.1007/s10664-023-10364-1>
45. Schölkopf, B., Platt, J.C., Shawe-Taylor, J., Smola, A.J., Williamson, R.C.: Estimating the support of a high-dimensional distribution. *Neural Comput.* 13(7), 1443–1471 (2001). <https://doi.org/10.1162/089976601750264965>
46. Alghushairy, O., Alsini, R., Soule, T., Ma, X.: A review of local outlier factor algorithms for outlier detection in big data streams. *Big Data Cogn. Comput.* 5(1), 1 (2020). <https://doi.org/10.3390/bdcc5010001>
47. Abhaya, Gupta, M., Patra, B.K.: Accelerating LOF outlier detection approach. In: Gupta, D., Goswami, R.S., Banerjee, S., Tanveer, M., Pachori, R.B. (eds.) *Pattern Recognition and Data Analysis with Applications*, pp. 191–203. Springer, Singapore (2022). https://doi.org/10.1007/978-981-19-1520-8_15
48. Shahapure, K.R., Nicholas, C.: Cluster quality analysis using silhouette score. In: *Proc. IEEE Int. Conf. on Data Science and Advanced Analytics (DSAA)*, pp. 747–748. IEEE (2020). <https://api.semanticscholar.org/CorpusID:227122930>
49. Rousseeuw, P.J.: Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *J. Comput. Appl. Math.* 20, 53–65 (1987). [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)
50. Qamar, U., Niza, R., Bashir, S., Khan, F.H.: A majority vote based classifier ensemble for web service classification. *Bus. Inf. Syst. Eng.* 58(4), (2016). <https://dl.gi.de/handle/20.500.12116/10690>
51. Alexandropoulos, S.-A.N., Aridas, C.K., Kotsiantis, S.B., Vrahatis, M.N.: Stacking strong ensembles of classifiers. In: *Artificial Intelligence Applications and Innovations*, pp. 545–556. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-19823-7_46
52. Louppe, G.: Understanding random forests: from theory to practice. *CoRR abs/1407.7502* (2014). <https://api.semanticscholar.org/CorpusID:88520060>
53. Zhu, J., He, S., He, P., Liu, J., Lyu, M. R.: Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In: *Proceedings of the 34th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pp. 355–366 (2023). <https://doi.org/10.1109/ISSRE59848.2023.00071>
54. Liashchynskiy, P.B., Liashchynskiy, P.B.: Grid search, random search, genetic algorithm: a big comparison for NAS. *CoRR abs/1912.06059* (2019). <https://api.semanticscholar.org/CorpusID:209323928>
55. Sipper, M.: High per parameter: a large-scale study of hyperparameter tuning for machine learning algorithms. *Algorithms* 15(9), 315 (2022). <https://api.semanticscholar.org/CorpusID:250493089>
56. Probst, P., Boulesteix, A.-L., Bischl, B.: Tunability: importance of hyperparameters of machine learning algorithms. *J. Mach. Learn. Res.* 20(1), 1934–1965 (2019). <https://api.semanticscholar.org/CorpusID:88515435>

57. Guo, H., Yuan, S., Wu, X.: LogBERT: log anomaly detection via BERT. In: Proc. Int. Joint Conf. on Neural Networks (IJCNN), pp. 1–8. IEEE (2021). <https://api.semanticscholar.org/CorpusID:232146842>
58. Wang, M., Zheng, B., Wang, G., Yang, J., Lu, J., Zhang, W.: A principal component analysis-based feature optimization network for few-shot fine-grained image classification. *Mathematics* **13**(7), Article 1098 (2025). <https://doi.org/10.3390/math13071098>
59. Liu, S., Deng, L., Xu, H., Wang, W.: LogBD: A log anomaly detection method based on pretrained models and domain adaptation. *Applied Sciences* **13**(13), Article 7739 (2023). <https://doi.org/10.3390/app13137739>
60. Lv, D., Luktarhan, N., Chen, Y.: ConAnomaly: Content-Based Anomaly Detection for System Logs. *Sensors* **21**(18), Article 6125 (2021). <https://doi.org/10.3390/s21186125>
61. Hundman, K., Constantinou, V., Laporte, C., Colwell, I., Soderstrom, T.: Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding. In: Proc. ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining, pp. 387–395. ACM (2018). <https://doi.org/10.1145/3219819.3219845>
62. Zaheer, M., Kottur, S., Ravanbakhsh, S., Póczos, B., Salakhutdinov, R., Smola, A.J.: Deep sets. In: Proc. Int. Conf. on Neural Information Processing Systems (NeurIPS), pp. 3394–3404 (2017). <https://api.semanticscholar.org/CorpusID:4870287>
63. Khan, S.W., Hafeez, Q., Khalid, M.I., Alroobaea, R., Hussain, S., Iqbal, J., Almotiri, J., Ullah, S.S.: Anomaly detection in traffic surveillance videos using deep learning. *Sensors* **22**(17), 6563 (2022). <https://doi.org/10.3390/s22176563>
64. Chen, T., Guestrin, C.: XGBoost: A Scalable Tree Boosting System. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794 (2016). <https://doi.org/10.1145/2939672.2939785>